Experiment 12: Machine Learning Lifecycle and Deployment using MLflow

Experiment No.	12	Mapped CO	CO5
Subject	Agile Software Development and DevOps Lab	Course Code	2175119

Aim

To track machine-learning experiments, version models and deploy a model using MLflow.

Tools / Software

Python 3.10 or above; MLflow; scikit-learn; sample dataset; terminal / command prompt; web browser.

Theory

MLOps applies DevOps principles to the machine-learning lifecycle, including data preparation, model training, evaluation, deployment and monitoring. A major challenge in machine learning is reproducibility because the final result depends on the dataset, source code, parameters and execution environment.

MLflow supports reproducible machine-learning workflows through four major components: Tracking, Projects, Models and Model Registry. MLflow Tracking records parameters, metrics and artifacts for every run. The Model Registry supports model versioning and stage transitions such as Staging and Production. The selected model can be served behind a REST endpoint for prediction requests.

Procedure

1. Install MLflow and required Python libraries such as scikit-learn.
2. Start the MLflow tracking UI using the mlflow ui command.
3. Create a training script and start an MLflow run in the script.
4. Train a machine-learning model and log hyperparameters, metrics and model artifacts.
5. Compare runs in the MLflow UI and identify the best run based on evaluation metrics.
6. Register the best model in the MLflow Model Registry.
7. Serve the registered or selected model using mlflow models serve and send a prediction request.
8. Paste screenshots or copied outputs in the observation boxes provided below.

Sample Code / Commands

train.py - MLflow tracking sample

```
import mlflow
import mlflow.sklearn
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

with mlflow.start_run():
    model = RandomForestClassifier(n_estimators=100)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
```

```
    mlflow.log_param('n_estimators', 100)
    mlflow.log_metric('accuracy', acc)
    mlflow.sklearn.log_model(model, 'model')
```

MLflow UI and model serving commands

```
mlflow ui
mlflow models serve -m runs:/<run_id>/model -p 1234 --no-conda

# Example prediction request after serving
curl -X POST http://127.0.0.1:1234/invocations \
  -H "Content-Type: application/json" \
  -d '{"dataframe_split":{"columns":["f1","f2"],"data":[[1,2]]}}'
```

Observation / Experiment Output

Instructions to students: Paste screenshots or copied outputs in the boxes below after completing MLflow installation, experiment tracking, model registration and model serving. Keep screenshots clear and readable.

Name	Roll No.	Batch	Date
Gargee Warke	65	B2	16-10-26

1. MLflow Installation and Tracking UI Output

Paste screenshot/output showing successful MLflow installation and the MLflow tracking UI running in the browser.

```
(venv) PS D:\mlflow-lab exp10> python -m pip install --upgrade pip
>> python -m pip install mlflow scikit-learn pandas numpy
Downloading urllib3-2.8.0-py3-none-any.whl (135 kB)
Downloading uvicorn-0.54.0-py3-none-any.whl (87 kB)
Downloading waitress-3.0.2-py3-none-any.whl (56 kB)
Downloading yarl-1.25.1-cp312-cp312-win_amd64.whl (102 kB)
Downloading aiohappyeyeballs-2.7.1-py3-none-any.whl (15 kB)
Downloading aiosignal-1.4.0-py3-none-any.whl (7.5 kB)
Downloading annotated_doc-0.0.5-py3-none-any.whl (5.3 kB)
Downloading annotated_types-0.8.0-py3-none-any.whl (13 kB)
Downloading attrs-26.1.0-py3-none-any.whl (67 kB)
Using cached blinker-1.9.0-py3-none-any.whl (8.5 kB)
Downloading certifi-2026.10.18-py3-none-any.whl (134 kB)
Downloading cffi-2.1.1-cp312-cp312-win_amd64.whl (185 kB)
Downloading contourpy-1.4.0-cp312-cp312-win_amd64.whl (234 kB)
Downloading cycler-0.12.1-py3-none-any.whl (8.3 kB)
Downloading fonttools-4.67.0-cp312-cp312-win_amd64.whl (2.5 MB)
   ---------------------------------------- 2.5/2.5 MB 11.9 MB/s  0:00:00
Downloading frozenlist-1.8.0-cp312-cp312-win_amd64.whl (44 kB)
Downloading h11-0.16.0-py3-none-any.whl (37 kB)
Using cached itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Using cached jinja2-3.1.6-py3-none-any.whl (134 kB)
Downloading joblib-1.6.0-py3-none-any.whl (306 kB)
Downloading kiwisolver-1.5.1-cp312-cp312-win_amd64.whl (70 kB)
Using cached markupsafe-3.0.4-cp312-cp312-win_amd64.whl (14 kB)
Downloading narwhals-2.27.1-py3-none-any.whl (483 kB)
Downloading pillow-13.0.0-cp312-cp312-win_amd64.whl (7.3 MB)
   ---------------------------------------- 7.3/7.3 MB 6.2 MB/s  0:00:01
Downloading prettytable-3.18.0-py3-none-any.whl (37 kB)
Downloading propcache-0.5.4-cp312-cp312-win_amd64.whl (46 kB)
Downloading pyasn1_modules-0.4.2-py3-none-any.whl (181 kB)
Downloading pyasn1-0.6.4-py3-none-any.whl (84 kB)
Downloading pyparsing-3.3.3-py3-none-any.whl (126 kB)
Downloading pywin32-312-cp312-cp312-win_amd64.whl (6.9 MB)
   ---------------------------------------- 6.9/6.9 MB 13.7 MB/s  0:00:00
Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)
Downloading threadpoolctl-3.7.0-py3-none-any.whl (26 kB)
Downloading typing_inspection-0.4.4-py3-none-any.whl (14 kB)
Downloading wcwidth-0.9.2-cp310-abi3-win_amd64.whl (597 kB)
   ---------------------------------------- 597.8/597.8 kB 22.4 MB/s  0:00:00
Using cached werkzeug-3.1.9-py3-none-any.whl (228 kB)
Downloading zipp-4.1.1-py3-none-any.whl (10 kB)
Downloading mako-1.4.3-py3-none-any.whl (80 kB)
Downloading pycparser-3.11-py3-none-any.whl (51 kB)
Downloading tzdata-2026.5-py2.py3-none-any.whl (347 kB)
Installing collected packages: huey, zipp, wcwidth, waitress, urllib3, tzdata, typing-extensions, threadpoolctl, sqlparse, smmap, six, pyyaml, pywin32, python-dotenv, pyparsing, pycparser, pyasn1, pyarrow, protobuf, propcache, pillow, packaging, numpy
, narwhals, multidict, markupsafe, kiwisolver, itsdangerous, idna, h11, graphql-core, frozenlist, fonttools, cycler, cloudpickle, click, charset_normalizer, certifi, cachetools, blinker, attrs, annotated-types, annotated-doc, aiohappyeyeballs, yarl, w
erkzeug, uvicorn, typing-inspection, sqlalchemy, scipy, requests, python-dateutil, pydantic-core, pyasn1-modules, prettytable, opentelemetry-proto, opentelemetry-api, Mako, joblib, jinja2, importlib_metadata, graphql-relay, gitdb, contourpy, cffi, any
io, aiosignal, starlette, scikit-learn, pydantic, pandas, opentelemetry-semantic-conventions, matplotlib, graphene, gitpython, Flask, docker, cryptography, alembic, aiohttp, skops, opentelemetry-sdk, google-auth, Flask-CORS, fastapi, databricks-sdk, m
lflow-tracing, mlflow-skinny, mlflow
Successfully installed Flask-3.1.3 Flask-CORS-6.0.5 Mako-1.4.3 aiohappyeyeballs-2.7.1 aiohttp-3.14.5 aiosignal-1.4.0 alembic-1.20.0 annotated-doc-0.0.5 annotated-types-0.8.0 anyio-4.15.1 attrs-26.1.0 blinker-1.9.0 cachetools-7.2.3 certifi-2026.10.18 c
ffi-2.1.1 charset_normalizer-3.5.2 click-8.5.0 cloudpickle-3.1.2 contourpy-1.4.0 cryptography-50.0.2 cycler-0.12.1 databricks-sdk-0.154.0 docker-7.2.0 fastapi-0.148.0 fonttools-4.67.0 frozenlist-1.8.0 gitdb-4.0.12 gitpython-3.2.1 google-auth-2.62.0 gr
aphene-3.4.3 graphql-core-3.2.13 graphql-relay-3.2.0 h11-0.16.0 huey-3.4.0 idna-3.20 importlib_metadata-9.0.1 itsdangerous-2.2.0 jinja2-3.1.6 joblib-1.6.0 kiwisolver-1.5.1 markupsafe-3.0.4 matplotlib-3.11.2 mlflow-3.17.1 mlflow-skinny-3.17.1 mlflow-tr
acing-3.17.1 multidict-7.1.0 narwhals-2.27.1 numpy-2.5.4 opentelemetry-api-1.45.1 opentelemetry-proto-1.45.1 opentelemetry-sdk-1.45.1 opentelemetry-semantic-conventions-0.66b1 packaging-26.3 pandas-3.0.6 pillow-13.0.0 prettytable-3.18.0 propcache-0.5.
4 protobuf-7.36.2 pyarrow-25.0.1 pyasn1-0.6.4 pyasn1-modules-0.4.2 pycparser-3.11 pydantic-2.14.1 pydantic-core-2.50.1 pyparsing-3.3.3 python-dateutil-2.9.0.post0 python-dotenv-1.2.4 pywin32-312 pyyaml-6.0.3 requests-2.34.2 scikit-learn-1.9.1 scipy-1.
18.1 six-1.17.0 skops-0.16.0 smmap-5.0.3 sqlalchemy-2.1.5 sqlparse-0.6.0 starlette-1.9.0 threadpoolctl-3.7.0 typing-extensions-4.16.0 typing-inspection-0.4.4 tzdata-2026.5 urllib3-2.8.0 uvicorn-0.54.0 waitress-3.0.2 wcwidth-0.9.2 werkzeug-3.1.9 yarl-1
.25.1 zipp-4.1.1
(venv) PS D:\mlflow-lab exp10>
```

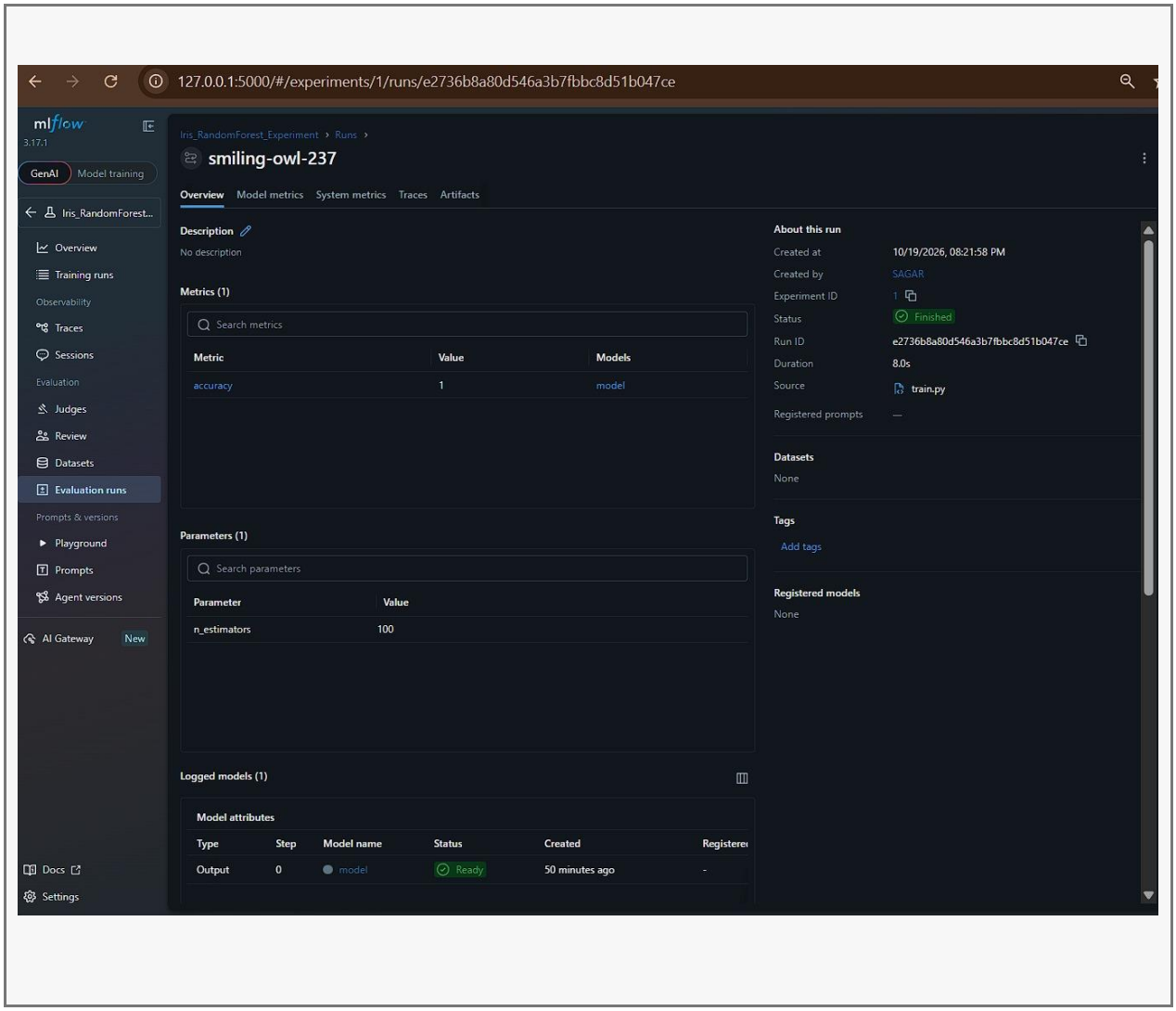
2. Training Script Execution Output

Paste screenshot/output showing execution of train.py and successful completion of a model training run.

```
(venv) PS D:\mlflow-lab exp10> python train.py
2026/10/19 20:21:59 WARNING mlflow.sklearn: Saving scikit-learn mod
rialization mechanism, which can execute arbitrary code during dese
.org/stable/model_persistence.html
Accuracy: 1.0
Run ID: e2736b8a80d546a3b7fbbc8d51b047ce
(venv) PS D:\mlflow-lab exp10>
```

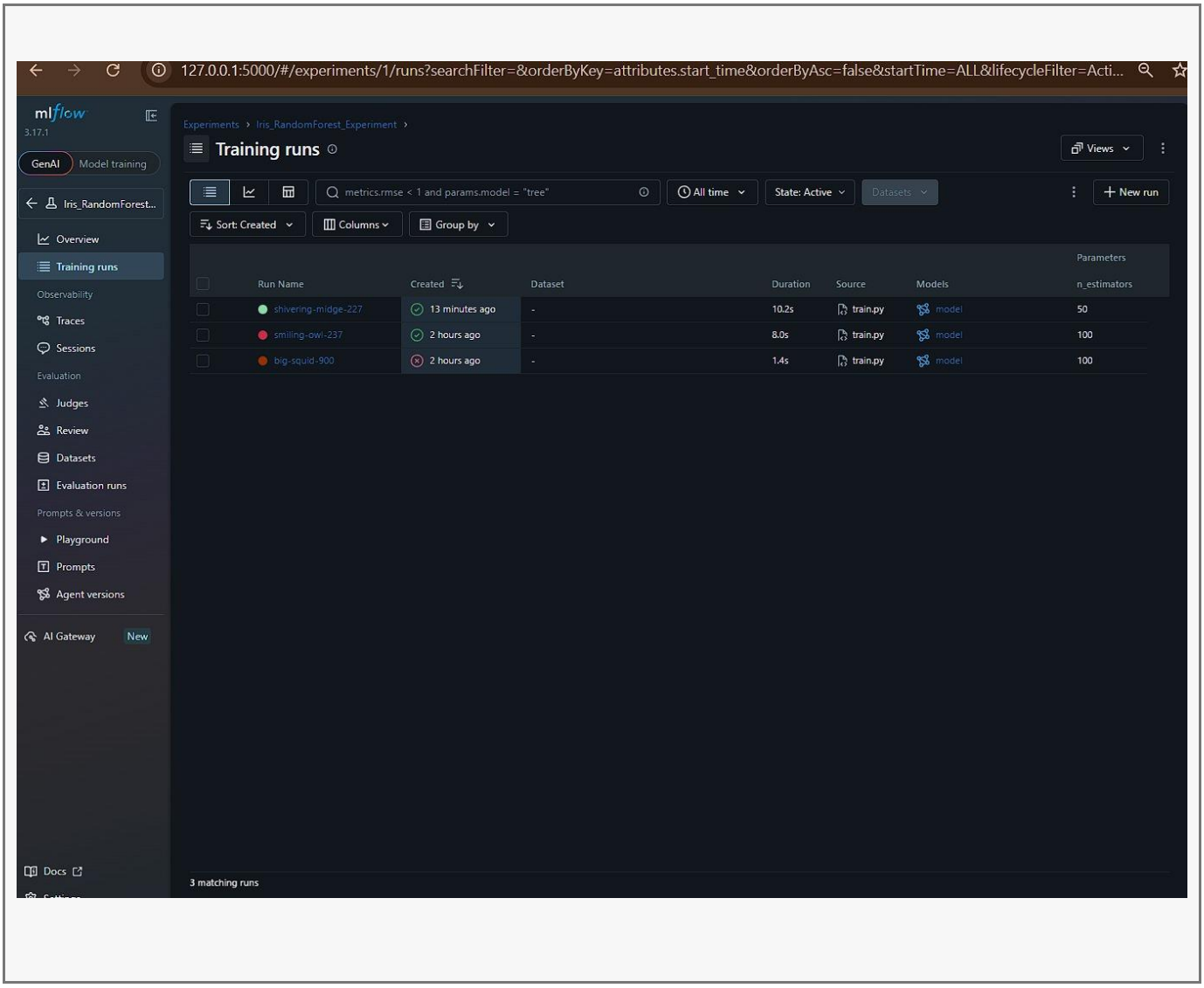
3. Parameters, Metrics and Artifacts Output

Paste screenshot/output showing logged parameters, accuracy or other metrics, and model artifact in the MLflow UI.



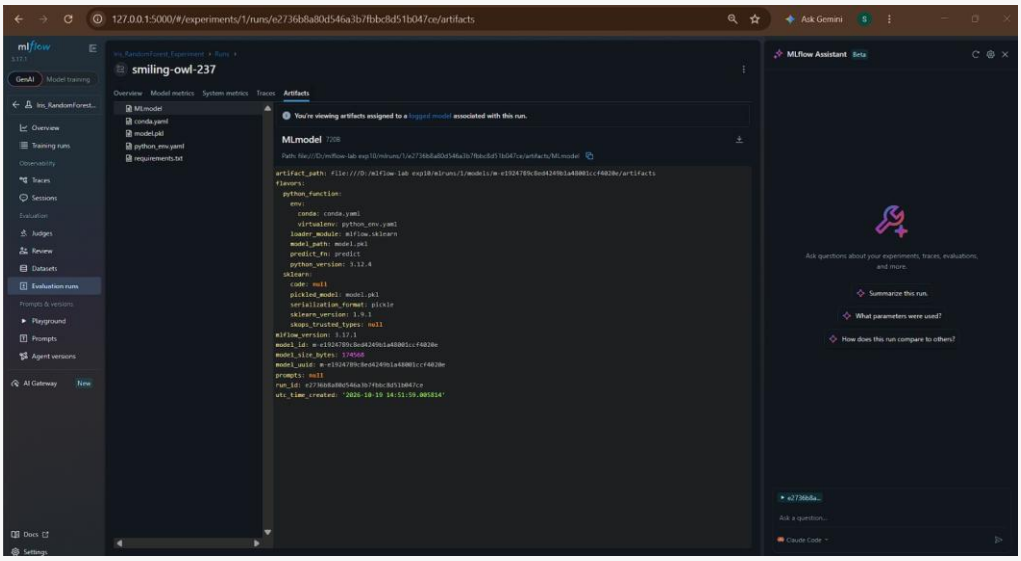
4. Run Comparison and Model Registry Output

Paste screenshot/output showing comparison of runs and registration of the best model in the Model Registry.



5. Model Serving and Prediction Output

Paste screenshot/output showing mlflow models serve command and prediction response from the served REST endpoint.



MLflow Tracking UI URL:

_____

Model Serving Endpoint URL:

_____

GitHub Repository / Training Script Link:

_____

Final Observation Statement

Write 2-3 lines summarizing the result of the experiment.

_____
_____
_____

Expected Output

Each training run appears in the MLflow UI with logged parameters, metrics and model artifact. Runs are comparable, the best model is registered, and the served endpoint returns predictions for posted input.

Conclusion

The machine-learning lifecycle was managed using MLflow by logging and comparing experiments, versioning a model in the registry and serving the model, demonstrating reproducible MLOps practice.

Viva / Review Questions

1. What problem does experiment tracking solve?

2. Name the four components of MLflow.
3. What is the purpose of the Model Registry?
4. How does MLOps differ from traditional DevOps?

Marks: ____ / 10	Date: _____	Signature of Faculty: _____